Vole Simulator - Antonuos's Part

ALU, Operation + 12 subclasses, ProgramLoader, InvalidInstructionException, InvalidProgramFileException

Operations (12)

Load (1RXY), LoadImmediate (2RXY), Store (3RXY), Move (40RS), AddInt (5RST), AddFloat (6RST), Or (7RST), And (8RST), Xor (9RST), Rotate (AR0X), Jump (BRXY), Halt (C000).

Interfaces assumed from teammates

- Machine: memory(), registers(), screen(), setProgramCounter(int), halt()
- Memory: read(int), write(int, uint8_t), size()
- RegisterFile: get(int), set(int, uint8_t)
- Screen: display(uint8_t)
- Instruction: getR(), getS(), getT(), getXY()
- CPU advances the PC by 2 before executing, so Jump only sets the PC

InvalidInstructionException.h

```
#pragma once
#include <stdexcept>
#include <string>

class InvalidInstructionException : public std::runtime_error {
public:
    explicit InvalidInstructionException(const std::string& message)
        : std::runtime_error("Invalid instruction: " + message) {}
};
```

InvalidProgramFileException.h

```
#pragma once
#include <stdexcept>
#include <string>

class InvalidProgramFileException : public std::runtime_error {
public:
    explicit InvalidProgramFileException(const std::string& message)
        : std::runtime_error("Invalid program file: " + message) {}
};
```

ALU.h

```
#pragma once
#include <cstdint>

// Stateless arithmetic/logic unit. All values are 8-bit register contents.
class ALU {
public:
    static uint8_t addTwosComplement(uint8_t a, uint8_t b);
    static uint8_t addFloat(uint8_t a, uint8_t b);
    static uint8_t bitOr(uint8_t a, uint8_t b);
    static uint8_t bitAnd(uint8_t a, uint8_t b);
    static uint8_t bitXor(uint8_t a, uint8_t b);
    static uint8_t rotateRight(uint8_t value, int count);

    // 8-bit float: sign(1) | exponent(3, excess-4) | mantissa(4, 0.1xxx normalized)
    static double  decodeFloat(uint8_t bits);
    static uint8_t encodeFloat(double value);
};
```

ALU.cpp

```
#include "ALU.h"
#include <cmath>

uint8_t ALU::addTwosComplement(uint8_t a, uint8_t b) {
    return static_cast<uint8_t>(a + b);  // wraps modulo 256 (overflow discarded)
}

uint8_t ALU::bitOr(uint8_t a, uint8_t b)  { return a | b; }
uint8_t ALU::bitAnd(uint8_t a, uint8_t b) { return a & b; }
uint8_t ALU::bitXor(uint8_t a, uint8_t b) { return a ^ b; }

uint8_t ALU::rotateRight(uint8_t value, int count) {
    count %= 8;
    if (count == 0) return value;
    return static_cast<uint8_t>((value >> count) | (value << (8 - count)));
}

double ALU::decodeFloat(uint8_t bits) {
    int sign = (bits & 0x80) ? -1 : 1;
    int exponent = ((bits >> 4) & 0x07) - 4;
    double mantissa = (bits & 0x0F) / 16.0;
    return sign * std::ldexp(mantissa, exponent);
}

uint8_t ALU::encodeFloat(double value) {
    if (value == 0.0 || std::isnan(value)) return 0x00;
    uint8_t sign = (value < 0) ? 0x80 : 0x00;
    if (std::isinf(value)) return sign | 0x7F;

    int exp = 0;
    double m = std::frexp(std::fabs(value), &exp);  // m in [0.5, 1)
    int stored = exp + 4;
    if (stored < 0) return 0x00;            // underflow -> 0
    if (stored > 7) return sign | 0x7F;     // overflow -> largest magnitude
    int mant = static_cast<int>(m * 16.0);  // truncate to 4 bits
    return static_cast<uint8_t>(sign | (stored << 4) | (mant & 0x0F));
}

uint8_t ALU::addFloat(uint8_t a, uint8_t b) {
    return encodeFloat(decodeFloat(a) + decodeFloat(b));
}
```

Operation.h

```
#pragma once
#include <string>

class Instruction;
class Machine;

// Base class for all 12 Vole operations (Command pattern).
class Operation {
public:
    virtual ~Operation() = default;
    virtual void execute(const Instruction& instruction, Machine& machine) const = 0;
    virtual std::string name() const = 0;
};
```

Operations.h

```
#pragma once
#include "Operation.h"

// 1RXY  R <- memory[XY]
class LoadOperation : public Operation {
public:
    void execute(const Instruction&, Machine&) const override;
    std::string name() const override { return "LOAD"; }
};
// 2RXY  R <- XY
class LoadImmediateOperation : public Operation {
public:
    void execute(const Instruction&, Machine&) const override;
    std::string name() const override { return "LOADI"; }
};
// 3RXY  memory[XY] <- R   (XY == 00 writes to the screen)
class StoreOperation : public Operation {
public:
    void execute(const Instruction&, Machine&) const override;
    std::string name() const override { return "STORE"; }
};
// 40RS  S <- R
class MoveOperation : public Operation {
public:
    void execute(const Instruction&, Machine&) const override;
    std::string name() const override { return "MOVE"; }
};
// 5RST  R <- S + T (two's complement)
class AddIntOperation : public Operation {
public:
    void execute(const Instruction&, Machine&) const override;
    std::string name() const override { return "ADD"; }
};
// 6RST  R <- S + T (floating point)
class AddFloatOperation : public Operation {
public:
    void execute(const Instruction&, Machine&) const override;
    std::string name() const override { return "ADDF"; }
};
// 7RST  R <- S OR T
class OrOperation : public Operation {
public:
    void execute(const Instruction&, Machine&) const override;
    std::string name() const override { return "OR"; }
};
// 8RST  R <- S AND T
class AndOperation : public Operation {
public:
    void execute(const Instruction&, Machine&) const override;
    std::string name() const override { return "AND"; }
};
// 9RST  R <- S XOR T
class XorOperation : public Operation {
public:
    void execute(const Instruction&, Machine&) const override;
    std::string name() const override { return "XOR"; }
};
// AR0X  rotate R right X times
class RotateOperation : public Operation {
public:
    void execute(const Instruction&, Machine&) const override;
    std::string name() const override { return "ROTATE"; }
};
// BRXY  if R == R0 then PC <- XY
class JumpOperation : public Operation {
public:
    void execute(const Instruction&, Machine&) const override;
    std::string name() const override { return "JUMP"; }
};
// C000  halt
class HaltOperation : public Operation {
public:
    void execute(const Instruction&, Machine&) const override;
```

```
    std::string name() const override { return "HALT"; }
};
```

Operations.cpp

```
#include "Operations.h"
#include "ALU.h"
#include "Instruction.h"
#include "Machine.h"

void LoadOperation::execute(const Instruction& i, Machine& m) const {
    m.registers().set(i.getR(), m.memory().read(i.getXY()));
}

void LoadImmediateOperation::execute(const Instruction& i, Machine& m) const {
    m.registers().set(i.getR(), static_cast<uint8_t>(i.getXY()));
}

void StoreOperation::execute(const Instruction& i, Machine& m) const {
    uint8_t value = m.registers().get(i.getR());
    if (i.getXY() == 0x00) m.screen().display(value);
    else                   m.memory().write(i.getXY(), value);
}

void MoveOperation::execute(const Instruction& i, Machine& m) const {
    // format 0RS: nibble2 = source R, nibble3 = destination S
    m.registers().set(i.getT(), m.registers().get(i.getS()));
}

void AddIntOperation::execute(const Instruction& i, Machine& m) const {
    auto& r = m.registers();
    r.set(i.getR(), ALU::addTwosComplement(r.get(i.getS()), r.get(i.getT())));
}

void AddFloatOperation::execute(const Instruction& i, Machine& m) const {
    auto& r = m.registers();
    r.set(i.getR(), ALU::addFloat(r.get(i.getS()), r.get(i.getT())));
}

void OrOperation::execute(const Instruction& i, Machine& m) const {
    auto& r = m.registers();
    r.set(i.getR(), ALU::bitOr(r.get(i.getS()), r.get(i.getT())));
}

void AndOperation::execute(const Instruction& i, Machine& m) const {
    auto& r = m.registers();
    r.set(i.getR(), ALU::bitAnd(r.get(i.getS()), r.get(i.getT())));
}

void XorOperation::execute(const Instruction& i, Machine& m) const {
    auto& r = m.registers();
    r.set(i.getR(), ALU::bitXor(r.get(i.getS()), r.get(i.getT())));
}

void RotateOperation::execute(const Instruction& i, Machine& m) const {
    auto& r = m.registers();
    r.set(i.getR(), ALU::rotateRight(r.get(i.getR()), i.getT()));
}

void JumpOperation::execute(const Instruction& i, Machine& m) const {
    auto& r = m.registers();
    if (r.get(i.getR()) == r.get(0)) m.setProgramCounter(i.getXY());
}

void HaltOperation::execute(const Instruction&, Machine& m) const {
    m.halt();
}
```

ProgramLoader.h

```
#pragma once
#include <string>

class Memory;

class ProgramLoader {
public:
    // Reads hex digits (whitespace ignored, 4 digits per instruction) from the
    // file and writes the bytes into memory starting at startAddress.
    // Throws InvalidProgramFileException on any problem.
    static void load(const std::string& path, Memory& memory, int startAddress = 0x00);
};
```

ProgramLoader.cpp

```
#include "ProgramLoader.h"
#include "InvalidProgramFileException.h"
#include "Memory.h"
#include <cctype>
#include <cstdint>
#include <fstream>
#include <vector>

namespace {
int hexValue(char c) {
    if (c >= '0' && c <= '9') return c - '0';
    c = static_cast<char>(std::toupper(static_cast<unsigned char>(c)));
    if (c >= 'A' && c <= 'F') return c - 'A' + 10;
    return -1;
}
}

void ProgramLoader::load(const std::string& path, Memory& memory, int startAddress) {
    std::ifstream file(path);
    if (!file) throw InvalidProgramFileException("cannot open '" + path + "'");

    std::vector<int> nibbles;
    char c;
    while (file.get(c)) {
        if (std::isspace(static_cast<unsigned char>(c))) continue;
        int v = hexValue(c);
        if (v < 0)
            throw InvalidProgramFileException(std::string("non-hex character '") + c + "'");
        nibbles.push_back(v);
    }

    if (nibbles.empty())
        throw InvalidProgramFileException("file is empty");
    if (nibbles.size() % 4 != 0)
        throw InvalidProgramFileException("each instruction must be exactly 4 hex digits");

    int byteCount = static_cast<int>(nibbles.size() / 2);
    if (startAddress < 0 || startAddress + byteCount > memory.size())
        throw InvalidProgramFileException("program does not fit in memory");

    for (int b = 0; b < byteCount; ++b)
        memory.write(startAddress + b,
                     static_cast<uint8_t>((nibbles[2 * b] << 4) | nibbles[2 * b + 1]));
}
```